

Computing pseudorandom functions within Ren

Field Extension Institute

RW10, 9th August 2026

Motivation

- ▶ Sources of (psuedo)randomness are essential to cryptography libraries.

¹* strictly speaking, we're using a pseudorandom number generator instead of a true PRF

Motivation

- ▶ Sources of (psuedo)randomness are essential to cryptography libraries.
- ▶ They also enable the class of randomized algorithms (albeit costly).

¹* strictly speaking, we're using a pseudorandom number generator instead of a true PRF

Motivation

- ▶ Sources of (psuedo)randomness are essential to cryptography libraries.
- ▶ They also enable the class of randomized algorithms (albeit costly).
- ▶ Plaintext-generated and public-key-encrypted randomness may not be viable (eg. untrusted server or other party).

¹* strictly speaking, we're using a pseudorandom number generator instead of a true PRF

Motivation

- ▶ Sources of (psuedo)randomness are essential to cryptography libraries.
- ▶ They also enable the class of randomized algorithms (albeit costly).
- ▶ Plaintext-generated and public-key-encrypted randomness may not be viable (eg. untrusted server or other party).

¹* strictly speaking, we're using a pseudorandom number generator instead of a true PRF

Motivation

- ▶ Sources of (psuedo)randomness are essential to cryptography libraries.
- ▶ They also enable the class of randomized algorithms (albeit costly).
- ▶ Plaintext-generated and public-key-encrypted randomness may not be viable (eg. untrusted server or other party).

Specific use case: rngeo, an FHE-based location game where scoring was based on a random number generator seeded by location and timestamp¹, among other things.

¹* strictly speaking, we're using a pseudorandom number generator instead of a true PRF

The problem

Can we efficiently (in depth, compute, memory use) implement a pseudorandom function as a circuit in Ren which takes encrypted inputs?

Common PRFs and barriers we observed

- ▶ Hash-based: hash a counter and secret seed to get a pseudorandom output. Something like SHA-256, but that requires $\text{mod-}2^{32}$ addition which is tricky whether we encode inputs as ints or bitstrings.

Common PRFs and barriers we observed

- ▶ Hash-based: hash a counter and secret seed to get a pseudorandom output. Something like SHA-256, but that requires $\text{mod-}2^{32}$ addition which is tricky whether we encode inputs as ints or bitstrings.
- ▶ Cipher-based: either block or stream ciphers, generating a sequence of pseudorandom outputs. AES has the SubBytes step which is a byte-substitution lookup table (8 depth for a polynomial that evaluates the LUT), and ChaCha also requires $\text{mod-}2^{32}$ addition so neither are particularly convenient to implement.

Some FHE-friendly PRF constructions

- ▶ Simple substitution-permutation networks (SPNs) which use FHE-friendly substitution (eg. polynomials) and permutation (eg. rotations) layers. This did not achieve very good confusion/diffusion metrics - Shannon's properties of secure ciphers.

Some FHE-friendly PRF constructions

- ▶ Simple substitution-permutation networks (SPNs) which use FHE-friendly substitution (eg. polynomials) and permutation (eg. rotations) layers. This did not achieve very good confusion/diffusion metrics - Shannon's properties of secure ciphers.
- ▶ Rasta (eprint 2018/181) is a stream cipher using alternating substitution and affine transforms over a binary bitstream. It is designed to have low AND-depth and use few ANDs per bit, with the only other operation being XOR (both of which are relatively simple in Ren).

Some FHE-friendly PRF constructions

- ▶ Simple substitution-permutation networks (SPNs) which use FHE-friendly substitution (eg. polynomials) and permutation (eg. rotations) layers. This did not achieve very good confusion/diffusion metrics - Shannon's properties of secure ciphers.
- ▶ Rasta (eprint 2018/181) is a stream cipher using alternating substitution and affine transforms over a binary bitstream. It is designed to have low AND-depth and use few ANDs per bit, with the only other operation being XOR (both of which are relatively simple in Ren).
- ▶ HERA (eprint 2020/135) was proposed as part of a CKKS/FV transciphering framework, and is a stream cipher on $\mathbb{Z}_t$. It is more performant than Rasta, but requires evaluating operations over a general modulus t .

Current status

rngео uses a Rasta-like PRF implementation on a 256-bit input; the same substitution layer as Rasta, but has a different affine layer (Rasta's uses a matrix multiplication).

Current status

rngео uses a Rasta-like PRF implementation on a 256-bit input; the same substitution layer as Rasta, but has a different affine layer (Rasta's uses a matrix multiplication).

Current numbers: 256-bit input/output, 4 rounds of alternating affine/subst. followed by a final affine, 8 rotation keys (at rngео secure params, ≈ 715 MB per key), 6 depth per round. Diffusion per bit-flip ≈ 0.39 , per-slot and pooled digit distributions within 15% of uniform.

Current status

rngео uses a Rasta-like PRF implementation on a 256-bit input; the same substitution layer as Rasta, but has a different affine layer (Rasta's uses a matrix multiplication).

Current numbers: 256-bit input/output, 4 rounds of alternating affine/subst. followed by a final affine, 8 rotation keys (at rngео secure params, ≈ 715 MB per key), 6 depth per round. Diffusion per bit-flip ≈ 0.39 , per-slot and pooled digit distributions within 15% of uniform.

Potential holes: the round constants for each affine layer are still generated server-side in plaintext.

Potential work directions

- ▶ Implement one of the common PRFs: involves coding up the missing components (eg. 32-bit adders or AES' SubBytes LUT)
- ▶ Optimization of existing circuit: how can we make the current implementation(s) faster, lower depth, etc?
- ▶ Search for other potential PRFs: what other PRFs can potentially be used within FHE?